



TaxWijs

Production Deployment Guide

Complete Step-by-Step Guide -- Zero to Production

Version 1.0 | May 2026

Django 6 + React 19 + Docker Compose + PostgreSQL

This document is confidential and intended for the technical operator of TaxWijs.

Keep this file secure -- it contains architecture details and deployment steps.

Table of Contents

Section 1	Architecture Overview
Section 2	What You Need Before Starting (Prerequisites)
Section 3	Create & Configure Your Server
Section 4	Install Docker on the Server
Section 5	Set Up Domain & SSL Certificate
Section 6	Clone the Repository & Configure Environment
Section 7	Deploy with Docker Compose
Section 8	Post-Deployment Setup
Section 9	Verify Everything Works
Section 10	Monitoring, Logs & Backups
Section 11	Troubleshooting Guide
Section 12	Annual Maintenance Checklist
Quick Reference	Commands, URLs & Checklist

i Time estimate: Setting up a fresh server and deploying takes approximately 2-3 hours the first time.

Section 1 -- Architecture Overview

TaxWijs is a production web application with three main components running inside Docker containers. All three containers communicate over a private internal Docker network. Only the frontend (nginx) is exposed to the internet on port 80 and 443. The database and backend API are never directly reachable from outside.

How the Three Containers Work Together

1. PostgreSQL (db): The database. Stores user accounts, tax rules, conversations, and all application data. It runs entirely inside the Docker network -- no outside access.
2. Django Backend (backend): The API server. Handles authentication, tax calculations, AI chat, and admin. Listens on port 8000 inside the Docker network only. Nginx forwards API requests to it.
3. React Frontend (frontend): A pre-built static React app served by nginx. Also acts as a reverse proxy, routing any request starting with /api/ to the backend container.

Request Flow Diagram

```
User Browser
|
|  HTTPS (port 443)  -- SSL terminated by nginx
v
[nginx inside frontend container]
|
|-- /api/* requests --> [Django backend:8000]  --> [PostgreSQL:5432]
|
+-- /* (everything else) --> React SPA (static files)
```

Why Docker Compose?

- * One command to start all three services: `docker compose up -d`
- * One command to stop everything: `docker compose down`
- * Environment variables (secrets, API keys) are isolated in a single `.env` file
- * The database data is stored in a named Docker volume -- it persists across restarts and updates
- * Easy to upgrade: `git pull`, `docker compose build`, `docker compose up -d`

v The database port (5432) and backend port (8000) are NOT exposed to the internet. This is intentional and is a security feature.

Section 2 -- Prerequisites

Before you touch any server, make sure you have all of the following ready.

2.1 -- A VPS (Virtual Private Server)

You need a cloud server. Below are the recommended providers and the minimum specifications for running TaxWijs smoothly:

Provider	Plan Name	RAM	CPU	SSD	Monthly Cost	Notes
DigitalOcean	Basic Droplet	2 GB	1 vCPU	50 GB	~\$12/mo	Recommended for start
Hetzner Cloud	CX22	4 GB	2 vCPU	40 GB	~\$6/mo	Best value in EU
Vultr	Cloud Compute	2 GB	1 vCPU	55 GB	~\$12/mo	Many data centers
AWS Lightsail	Standard	2 GB	1 vCPU	60 GB	~\$10/mo	AWS ecosystem

! Choose Ubuntu 22.04 LTS as the operating system. This guide is written specifically for Ubuntu.

2.2 -- A Domain Name

You need a domain name like taxwijs.nl or app.taxwijs.nl. Purchase from a registrar (Namecheap, TransIP, GoDaddy). Cost is typically EUR10-15/year. You will point the domain's DNS records to your server's IP address in Step 5.

2.3 -- API Keys You Need

Service	What It's For	Where to Get It	Required?
Anthropic (Claude)	AI chat responses	console.anthropic.com	YES
OpenAI	Text embeddings for RAG search	platform.openai.com	YES
Stripe	Payment processing (pricing page)	dashboard.stripe.com	YES
Stripe Webhook Secret	Verify Stripe events	Stripe dashboard > Webhooks	YES
Reddit API	Forum data scraping	reddit.com/prefs/apps	Optional

! Without Anthropic and OpenAI API keys, the chat feature and search will not work. Get these first.

2.4 -- Software on Your Local Machine

- * Git -- to clone the repository
- * An SSH client -- Terminal on Mac/Linux, or PuTTY / Windows Terminal on Windows
- * A text editor -- VS Code, Notepad++, or even nano on the server

Section 3 -- Create & Configure Your Server

1 Create the server

Go to your VPS provider (e.g., DigitalOcean), click Create > Droplet (or equivalent). Choose: Ubuntu 22.04 LTS, 2 GB RAM minimum, the data center closest to your users (e.g., Amsterdam for Dutch users). Add your SSH key when prompted (recommended) or note the root password they email you.

2 SSH into the server

Once the server is created, you'll see an IP address like 178.62.42.101. Open your terminal and connect:

```
ssh root@YOUR_SERVER_IP
```

If asked about fingerprint, type 'yes' and press Enter. You are now inside the server.

3 Create a non-root user

Running as root is dangerous for daily operations. Create a regular user:

```
adduser taxwijs
# Enter a strong password when prompted
# Press Enter to skip the other fields

# Give this user admin (sudo) privileges:
usermod -aG sudo taxwijs

# Switch to this new user:
su - taxwijs
```

All future commands in this guide should be run as the 'taxwijs' user, not root.

4 Update the server

Update all system packages (important for security):

```
sudo apt update && sudo apt upgrade -y
```

This may take 2-5 minutes. Install essential tools:

```
sudo apt install -y curl wget git ufw
```

5 Configure the firewall (UFW)

UFW (Uncomplicated Firewall) blocks all ports except the ones you allow. Enable it and open only what

is needed:

```
sudo ufw allow 22/tcp      # SSH (so you can connect)
sudo ufw allow 80/tcp      # HTTP (for Let's Encrypt verification)
sudo ufw allow 443/tcp     # HTTPS (your website)
sudo ufw enable            # Turn on the firewall
sudo ufw status            # Verify rules
```

v Never open port 5432 (PostgreSQL) or 8000 (Django) on the firewall. They stay private inside Docker.

Section 4 -- Install Docker on the Server

Docker is the engine that runs all three containers. Install it with these commands exactly as written:

6 Install Docker Engine

```
# Install Docker's official GPG key:
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add Docker repository:
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME")
stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker:
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

7 Allow your user to run Docker

Add your user to the docker group so you don't need sudo every time:

```
sudo usermod -aG docker taxwijs
newgrp docker

# Verify Docker works:
docker --version
docker compose version
```

You should see something like: Docker version 26.x.x and Docker Compose version v2.x.x

8 Enable Docker on startup

Make Docker start automatically when the server reboots:

```
sudo systemctl enable docker
sudo systemctl start docker
```

Section 5 -- Set Up Domain & SSL Certificate

5.1 -- Point Your Domain to the Server

Log into your domain registrar (where you bought the domain). Find the DNS management section. Create an A record pointing your domain to your server's IP:

Type	Name / Host	Value / Points To	TTL
A	@ (or taxwijs.nl)	YOUR_SERVER_IP	3600
A	www	YOUR_SERVER_IP	3600

DNS changes propagate in 5-60 minutes. You can check propagation at dnschecker.org.

! Wait until DNS propagates before running Certbot (Step 9). Certbot needs to reach your server via the domain.

9 Get a free SSL certificate with Certbot

Certbot installs Let's Encrypt SSL certificates for free. Install it:

```
sudo apt install -y certbot

# Get a certificate for your domain:
sudo certbot certonly --standalone -d yourdomain.nl -d www.yourdomain.nl

# Enter your email when prompted (for renewal notifications)
# Agree to terms of service: Y
# Decline to share email with EFF: N (optional)
```

If successful, your certificates are saved at:

```
/etc/letsencrypt/live/yourdomain.nl/fullchain.pem
/etc/letsencrypt/live/yourdomain.nl/privkey.pem
```

! Replace 'yourdomain.nl' with your actual domain name everywhere in this step.

10 Set up automatic SSL renewal

SSL certificates expire every 90 days. Set up automatic renewal:

```
# Test renewal works:
sudo certbot renew --dry-run

# Add automatic renewal via cron (runs twice daily):
sudo crontab -e
```



```
# Add this line at the bottom:  
0 12 * * * /usr/bin/certbot renew --quiet
```

Section 6 -- Clone Repository & Configure Environment

11 Clone the repository

Clone the TaxWijs repository onto the server:

```
cd /home/taxwijs
git clone https://github.com/YOUR_GITHUB_USERNAME/nl-tax-ai.git
cd nl-tax-ai
```

If your repository is private, you'll need to authenticate. Use a GitHub Personal Access Token (PAT) instead of your password. Create one at: GitHub > Settings > Developer Settings > Personal Access Tokens > Tokens (classic).

12 Create the .env file

The .env file contains all your secrets and configuration. This file is NEVER committed to git (it is in .gitignore). You create it manually on the server:

```
nano .env
```

Copy and paste the following template into the file. Fill in every value marked with <...>:

Complete .env Template

```
# -- Django Core -----
SECRET_KEY=<generate a 50+ char random string -- see note below>
DEBUG=False
ALLOWED_HOSTS=yourdomain.nl,www.yourdomain.nl
SECURE_SSL_REDIRECT=True

# -- Database -----
DATABASE_URL=postgres://taxwijs_user:STRONG_DB_PASSWORD@db:5432/taxwijs_db
POSTGRES_DB=taxwijs_db
POSTGRES_USER=taxwijs_user
POSTGRES_PASSWORD=<STRONG_DB_PASSWORD -- at least 20 characters>

# -- AI API Keys -----
ANTHROPIC_API_KEY=<sk-ant-api...>
OPENAI_API_KEY=<sk-proj-...>

# -- Payments -----
```

```
STRIPE_PUBLIC_KEY=<pk_live_...>
STRIPE_SECRET_KEY=<sk_live_...>
STRIPE_WEBHOOK_SECRET=<whsec_...>

# -- Email (optional -- for password reset emails) -----
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_HOST_USER=<your@email.nl>
EMAIL_HOST_PASSWORD=<app password>
DEFAULT_FROM_EMAIL=noreply@yourdomain.nl

# -- CORS -----
CORS_ALLOWED_ORIGINS=https://yourdomain.nl,https://www.yourdomain.nl

# -- Frontend Build -----
VITE_API_BASE_URL=https://yourdomain.nl
VITE_STRIPE_PUBLIC_KEY=<pk_live_...>
```

How to Generate a Secure SECRET_KEY

Run this command on the server to generate a secure random key:

```
python3 -c "import secrets; print(secrets.token_urlsafe(50))"
```

Copy the output and paste it as your SECRET_KEY value.

X The SECRET_KEY must be different on every deployment environment. Never reuse development keys in production.

Update Nginx Config with Your Domain

The nginx config needs your actual domain name. Edit the frontend Dockerfile or nginx config file and replace 'yourdomain.nl' with your real domain. In the project, find:

```
# File: frontend/nginx.conf (or similar)
server_name yourdomain.nl www.yourdomain.nl;
```

Also update the SSL certificate paths in nginx.conf to point to your Certbot certificate files.

Section 7 -- Deploy with Docker Compose

With the .env file ready and Docker installed, you are ready to deploy. These commands build all containers and start the entire application.

13 Build the Docker images

Build all Docker images. This downloads base images and installs all dependencies. The first time takes 5-15 minutes depending on internet speed:

```
cd /home/taxwijs/nl-tax-ai
docker compose build
```

You will see a lot of output as Docker downloads Python packages, Node modules, etc. This is normal. Wait until you see 'Successfully built' for each service.

14 Run database migrations

Run database migrations to create all database tables. This must be done BEFORE starting the full application:

```
docker compose run --rm backend python manage.py migrate
```

You should see output like:

```
Applying auth.0001_initial... OK
Applying users.0001_initial... OK
...
```

15 Collect static files

Collect all static files (Django admin CSS, etc.):

```
docker compose run --rm backend python manage.py collectstatic --noinput
```

16 Start the application

Start all three containers in detached mode (they run in the background):

```
docker compose up -d

# Check that all containers are running:
docker compose ps
```

All three services (db, backend, frontend) should show status 'running' or 'healthy'. If any show 'Exit' or 'Error', check logs with: `docker compose logs <service_name>`

Section 8 -- Post-Deployment Setup

After the containers are running, complete these essential setup steps.

17 Create the admin superuser

Create the first admin user who can log into /admin and manage the application:

```
docker compose exec backend python manage.py createsuperuser

# Enter:
# Username: admin (or your preferred name)
# Email: your@email.nl
# Password: <very strong password>
# Password (again): <same password>
```

Write down these credentials securely. This is the master admin account.

18 Build the search index (RAG)

The RAG search system requires building a vector index from tax rules. Run this once:

```
docker compose exec backend python phase2/build_index.py
```

This embeds all tax rules into ChromaDB. Takes about 1-2 minutes. You will see progress output as it processes each rule.

19 Register Stripe webhook

Stripe needs to send webhook events to your server when payments succeed. Register the webhook in your Stripe dashboard:

- * Log into dashboard.stripe.com
- * Go to: Developers > Webhooks > Add endpoint
- * Endpoint URL: <https://yourdomain.nl/api/payments/webhook/>
- * Select events: `payment_intent.succeeded,` `customer.subscription.created,`
`customer.subscription.deleted`
- * Click 'Add endpoint'
- * Copy the 'Signing secret' (starts with `whsec_`) into your `.env` as `STRIPE_WEBHOOK_SECRET`
- * Restart the backend: `docker compose restart backend`

20 Load seed data

Load the initial tax rules into the database (if not already done by migrations):

```
docker compose exec backend python manage.py loaddata
phase1/data/seed/tax_rules_2026.json
```

Section 9 -- Verify Everything Works

Run through this checklist completely before announcing the site is live.

9.1 -- Browser Checklist

#	What to Check	Expected Result
1	Open https://yourdomain.nl	Landing page loads with TaxWijs logo
2	Check browser padlock (SSL)	Shows 'Connection is secure'
3	Try http://yourdomain.nl	Should redirect to https:// automatically
4	Open /register	Registration form appears, submit works
5	Open /login	Login works with the account you just created
6	Open /chat after login	Chat interface loads, send a message
7	Open /intake	Intake form loads and steps through correctly
8	Open /pricing	Pricing page loads with plan cards
9	Open /dashboard after login	User dashboard shows session history
10	Open /admin with superuser	Django admin panel loads

9.2 -- API Health Checks (Run on the Server)

SSH into the server and run these curl commands to verify the API responds correctly:

```
# Health check endpoint:
curl https://yourdomain.nl/api/users/health/
# Expected: {"status": "ok"}

# Login endpoint (replace with your credentials):
curl -X POST https://yourdomain.nl/api/users/token/ \
  -H 'Content-Type: application/json' \
  -d '{"username": "admin", "password": "your_password"}'
# Expected: {"access": "...", "refresh": "..."}

# Calculator endpoint (requires auth token from above):
curl -X POST https://yourdomain.nl/api/calculator/ \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer YOUR_ACCESS_TOKEN' \
  -d '{"income": 50000, "user_type": "zzp"}'
# Expected: JSON with tax calculations
```

9.3 -- Container Status Check

```
docker compose ps
# All should show: running or healthy

docker compose logs backend --tail=20
# Should show: Django startup messages, no ERROR lines

docker compose logs frontend --tail=20
# Should show: nginx started
```


Section 10 -- Monitoring, Logs & Backups

10.1 -- How to Read Logs

Logs are your first tool for debugging any problem in production.

Command	What It Shows
<code>docker compose logs backend</code>	All Django logs (API requests, errors)
<code>docker compose logs frontend</code>	nginx access log, 502 errors
<code>docker compose logs db</code>	PostgreSQL logs
<code>docker compose logs -f backend</code>	Live tail -- keeps streaming new logs
<code>docker compose logs backend --tail=50</code>	Last 50 lines only

10.2 -- Database Backup

Back up your database regularly. A weekly cron job is the minimum. Run this manually to create a backup right now:

```
# Create a backup:
docker compose exec db pg_dump -U taxwijs_user taxwijs_db > backup_$(date +%Y%m%d).sql

# To restore a backup:
cat backup_20260527.sql | docker compose exec -T db psql -U taxwijs_user taxwijs_db
```

Add this to a cron job to run automatically every week:

```
crontab -e
# Add this line:
0 2 * * 0 cd /home/taxwijs/nl-tax-ai && docker compose exec -T db pg_dump -U taxwijs_user
taxwijs_db > ~/backups/db_$(date +%Y%m%d).sql
```

! Copy backups to an external location (e.g., S3 bucket or another server). A backup on the same server is not a real backup.

10.3 -- How to Update the Application

When you push new code, update the production server with these commands:

```
cd /home/taxwijs/nl-tax-ai
git pull origin master          # Pull latest code
docker compose build            # Rebuild containers
docker compose run --rm backend python manage.py migrate # Run new migrations
docker compose up -d           # Restart with new images
```

Total downtime for an update: approximately 30-60 seconds while containers restart.

10.4 -- Check Server Disk and Memory

```
df -h          # Disk usage -- watch that it doesn't fill up
free -h        # Memory usage
docker system df # Docker image/volume space usage

# Clean up old Docker images to free space:
docker image prune -f
```

Section 11 -- Troubleshooting Guide

These are the most common problems you will encounter and exactly how to fix them.

Problem 1: Site shows 'Connection Refused' or blank page

This means nginx is not running or the containers are not up.

```
docker compose ps          # Check container status
docker compose logs frontend # Check nginx logs for errors
docker compose up -d        # Start containers if they're down
docker compose restart frontend # Restart just nginx
```

Problem 2: API returns 502 Bad Gateway

nginx is running but cannot reach the Django backend. Backend may have crashed.

```
docker compose logs backend --tail=30 # Look for Python errors
docker compose restart backend         # Restart the backend
# Check if database is accessible:
docker compose exec backend python manage.py check --database default
```

Problem 3: Database migration fails

If you see 'relation does not exist' or migration errors:

```
# Check which migrations have run:
docker compose exec backend python manage.py showmigrations

# Run migrations:
docker compose exec backend python manage.py migrate

# If completely broken, reset (WARNING: deletes all data):
docker compose down -v          # Remove volumes
docker compose up -d            # Fresh start
docker compose exec backend python manage.py migrate
```

X docker compose down -v deletes the database volume and ALL data. Only do this on a fresh setup, never on a server with real user data.

Problem 4: SSL certificate error

If the browser shows 'Certificate expired' or 'SSL_ERROR_RX_RECORD_TOO_LONG':

```
# Check certificate expiry date:
sudo certbot certificates

# Renew manually:
sudo certbot renew

# After renewal, restart nginx to load the new cert:
docker compose restart frontend
```

Problem 5: 'ALLOWED_HOSTS' error (400 Bad Request)

Django is refusing requests because the domain isn't in ALLOWED_HOSTS.

```
# Edit .env and add your domain:
nano .env

# Change:
ALLOWED_HOSTS=yourdomain.nl,www.yourdomain.nl

# Restart backend:
docker compose restart backend
```

Problem 6: Out of disk space

Docker containers and images accumulate over time. Clean up:

```
docker system prune -a      # Remove unused images, containers, networks
docker volume prune         # Remove unused volumes (careful!)
df -h /                     # Check disk usage again
```

! docker volume prune will NOT delete named volumes (your database). It only removes anonymous volumes. Still check before running.

Section 12 -- Annual Maintenance Checklist

TaxWijs serves Dutch tax rules. Tax rules change every year -- typically announced in September (Prinsjesdag) and effective from January 1. This maintenance must be completed each year.

September-December: Tax Year Update Cycle

Month	Task	Time Required
September	Monitor Belastingdienst for Prinsjesdag announcements	2 hours
October	Update phase1/data/seed/tax_rules_YYYY.json with new values	4 hours
October	Update CLAUDE.md with new tax figures for the new year	1 hour
November	Run phase1/scripts/validate.py -- must pass 100%	30 min
November	Rebuild RAG index: python phase2/build_index.py	15 min
November	Update tax calculator with new brackets and rates	3 hours
December	Deploy updated version to production	1 hour
January	Verify tax figures with a CPA before serving real users	2 hours

Monthly Tasks

- * Check disk usage: docker system df -- clean up if over 70%
- * Review error logs: docker compose logs backend | grep ERROR
- * Verify SSL certificate is valid: sudo certbot certificates
- * Download and store a database backup offsite
- * Check API usage and costs in Anthropic and OpenAI dashboards

When Tax Rules Change Mid-Year

Sometimes rules change outside the normal cycle (e.g., emergency legislation). When this happens:

- * Update the relevant rule in tax_rules_2026.json
- * Change verification_status to 'pending_review' until verified
- * Run validate.py to confirm no integrity errors
- * Rebuild the index and deploy
- * Add a note in PROGRESS.md with the date and rule ID changed

Quick Reference

Essential Commands

Command	What It Does
<code>docker compose up -d</code>	Start all containers in background
<code>docker compose down</code>	Stop all containers (data preserved)
<code>docker compose down -v</code>	Stop + delete all data volumes
<code>docker compose build</code>	Rebuild all images after code changes
<code>docker compose ps</code>	Show status of all containers
<code>docker compose logs -f backend</code>	Live log stream from Django
<code>docker compose exec backend python manage.py migrate</code>	Run new migrations
<code>docker compose exec backend python manage.py</code>	Create admin user
<code>docker compose restart backend</code>	Restart backend only
<code>git pull && docker compose build && docker compose up -d</code>	Full update in one line

Important URLs After Deployment

URL	What It Is
<code>https://yourdomain.nl</code>	Main website (landing page)
<code>https://yourdomain.nl/login</code>	User login
<code>https://yourdomain.nl/chat</code>	AI tax chat interface
<code>https://yourdomain.nl/admin</code>	Django admin panel (staff only)
<code>https://yourdomain.nl/api/users/health/</code>	API health check
<code>https://yourdomain.nl/admin/rules</code>	Tax rules management

Security Checklist

Item	Status to Achieve
DEBUG=False in .env	Must be False in production
SECRET_KEY is unique and random	50+ characters, never shared
Database not exposed to internet	Port 5432 not in docker-compose
Backend not exposed to internet	Port 8000 not in docker-compose
SSL certificate active	Browser shows padlock
Superuser password is strong	12+ chars, mixed case, symbols
Firewall allows only 22/80/443	<code>sudo ufw status</code>
Backups running automatically	Weekly cron job configured

v You are ready to go live when every item in this security checklist is checked and all browser tests in Section 9 pass.